

SECTION C

Practical scenario question. This section carries 30 marks

Instructions

To submit your project, create a new public GitHub repository and upload (push) your complete project files to the repository. Ensure that all source code, assets, and documentation are included and that the repository is accessible to the instructor. After successfully pushing your project, copy the repository URL and submit it through the provided Google Sheet form along with any required personal information. Before submitting, verify that the repository link is correct, the project has been fully uploaded, and the repository remains public until grading is complete.

Project form: <https://forms.gle/iNVWSozYFX2BoKqaA>

Full-Stack Project: "PropSpace" (Property Listing App)

1. Project Overview

PropSpace is a real-time web application where users can list, view, update, and delete properties for rent or sale. The application includes user authentication, dynamic search filters, a user dashboard to manage personal property portfolios, and user account management profile settings.

Tech Stack Options

- **Frontend:** React OR Angular. UI styled with any ui lib.
- **Backend:** Node.js with Express.
- **Database:** MongoDB (NoSQL).

2. Functional Requirements

A. User & Authentication Management

- **Registration & Hashing:** Users can sign up with a unique email, username, and secure password (passwords must be salted and hashed).

- **Authentication State:** Secure login generating a state mechanism (JWT JSON Web Tokens).
- **Protected Access:** Users cannot manage listings or profiles without presenting a valid, non-expired authorization token.

B. Account Management Dashboard

- **Profile Read/Write:** Registered users can update personal configuration metrics (e.g., Change profile name, contact phone number, upload avatar link).
- **Security Adjustments:** Ability to update the current account password safely by verifying the old password first.

C. Property Listings Management (CRUD)

- **Create:** Authenticated users can create a listing with title, description, price, location (city/country), property type (Apartment, House, Studio), and image URLs.
- **Read (Public & Private):**
 - *Public Feed:* Anyone (even unauthenticated guests) can view a feed of all properties and search/filter them by city or price range.
 - *Private Feed:* Authenticated users have a dedicated "My Listings" screen that displays only properties they authored.
- **Update:** Authors can modify details of their existing listings. Non-authors must be strictly blocked by the backend API from making modifications.
- **Delete:** Authors can permanently remove a listing from the marketplace.

3. Technical Specification & Code Layout Guidelines

A. Backend Architecture Best Practices

1. **Routes Layer:** Keeps endpoints clean. Merely handles parsing routing params (e.g., `req.params.id`), triggering middleware, and delegating execution.
2. **Controller / Service Layer:** Implements business validation rules, format transformations, and calculations.
3. **Data Repository Layer:** Interacts with the database directly using models.

B. Frontend State & Lifecycle Best Practices

1. **State Initialization:** Utilize lifecycle initializations to run network requests exactly once during a component mount.
2. **Memory Cleanups:** Implement necessary event listener or stream subscription tear-downs during component unmounting to prevent runtime memory leakages.

3. **Global Interceptors:** Handle outbound authentication tokens efficiently using global HTTP Interceptors or centralized API hook wrappers.

4. Comprehensive Evaluation Criteria

Assessors evaluating this project should look for evidence of clean production-grade design choices across the following criteria:

Frontend Evaluation Metrics

- **Component Modularity:** Are UI blocks broken down into reusable elements (e.g., `PropertyCard`, `FilterSidebar`, `InputField`), or is the whole view crammed into one massive script?
- **Asynchronous UX State Handlers:** Does the application properly handle edge states dynamically? It must display explicit visual feedback for *Loading...*, *Empty Results*, and network *Errors* gracefully instead of breaking the browser layout.
- **Form Validations:** Prevent submission of incomplete property details or broken data formats before hitting the network pipeline.
- **Client Routing:** Route path matching should protect internal dashboard layouts using route guards. Unauthenticated route redirects must seamlessly push clients back to the login terminal.

Backend Evaluation Metrics

- **RESTful API Compliance:** Resource routes should adhere to clean semantic structure using standard HTTP request verbs (GET `/api/properties`, POST `/api/properties`, PUT `/api/properties/:id`, DELETE `/api/properties/:id`).
- **Secure Route Guards:** The backend *must not* trust the client-side state blindly. Token validation and object-ownership validation should happen directly at the server level prior to allowing database queries.
- **Payload Sanitation and HTTP Status Mapping:** Outbound API channels must respond with precise status headers:
 - 200 OK / 201 Created for successful logic states.
 - 400 Bad Request for invalid schemas or malformed types.
 - 401 Unauthorized / 403 Forbidden for failed tokens or cross-user modifications.
 - 404 Not Found for nonexistent database lookups.
- **Database Schema Optimization:** Database schemas must use references or embedded attributes appropriately to link an individual property listing to a specific user account ID.

